Ecole Nationale d'Ingénieurs de Sfax Département de Génie Informatique et de Mathématiques Appliquées		Année universitaire 2025 - 2026

Matière : **Computer Vision 1**
Auditoire : **GI 1**
Responsable du cours : **Pr. Chokri BEN AMAR**
Enseignante des TPs : **Dr. Zeineb DAOUD**

TP 2 : Amélioration d'images

Manipulation d'histogrammes, seuillage, filtrage et transformations géométriques

Objectifs du TP

L'objectif principal de ce laboratoire est de concevoir et de développer une **interface graphique (GUI)** interactive dédiée à l'application de différents types d'amélioration d'images. Plus spécifiquement, les tâches de ce TP s'articulent autour de quatre axes :

- **Analyse et Amélioration :** Maîtriser le calcul et l'affichage d'histogrammes, ainsi que l'application de l'**égalisation** pour optimiser le contraste des images.
- **Binarisation et seuillage :** Mettre en œuvre différentes méthodes de **seuillage** pour extraire des informations pertinentes.
- **Filtrage Spatial :** Mettre en œuvre des filtres de convolution essentiels pour le traitement du bruit et l'amélioration des contours, notamment les filtres **moyen**, **médian**, et **gaussien**.
- **Opérations Géométriques :** Transformer une image via des fonctions de **rotation**, d'**extraction** (cropping) et d'**agrandissement**.

I. Concepts de base pour l'amélioration des images avec OpenCV

1. Égalisation d'histogramme

L'amélioration par histogramme vise à modifier la distribution des niveaux de gris pour améliorer le contraste ou la visibilité des détails.

❖ **Calcul de l'histogramme**

```
# hist est un tableau de 256 valeurs représentant le nombre de pixels pour chaque intensité
hist = cv2.calcHist([image], [0], None, [256], [0, 256])
```

L'amélioration par égalisation d'histogramme vise à distribuer uniformément les niveaux de gris pour améliorer le contraste ou la visibilité des détails.

- **Égalisation d'histogramme**

Cette opération permet de redistribuer uniformément les intensités de l'image sur toute la plage disponible (0-255). Elle est idéale pour les images trop sombres ou trop claires. La fonction **cv2.equalizeHist** effectue l'égalisation avec OpenCV.

```
# L'image d'entrée doit obligatoirement être en niveaux de gris
image_egalisee = cv2.equalizeHist(image_grise)
```

2. Seuillage (Thresholding)

Le seuillage est une opération de segmentation de l'image qui permet de séparer les objets d'intérêt (premier plan) du fond de l'image en créant une image binaire (Noir et Blanc).

Le seuillage d'une image s'effectue sous OpenCV en utilisant la fonction cv2.threshold.

- **Seuillage binaire (simple) :** On fixe un seuil ***T***. Tout pixel **>** ***T*** devient blanc (255), sinon noir (0).

```
ret, image_seuil = cv2.threshold(image, 120, 255, cv2.THRESH_BINARY)
```

<u>Paramètres de la fonction</u>

- **image** : C'est l'image source (généralement en niveaux de gris).
- **120 (Le seuil)** : Chaque pixel de l'image sera comparé à ce nombre.
  - Si le pixel est « **clair** » (supérieur à 120), il devient **blanc** (255).
  - Si le pixel est « **sombre** » (inférieur ou égal à 127), il devient **noir** (0).
- **255 (valeur max)** : C'est la valeur qui sera attribuée aux pixels qui dépassent le seuil. Ici, 255 correspond au blanc pur.
- **cv2.THRESH_BINARY** : C'est le mode de calcul. Il indique à OpenCV d'appliquer une logique binaire simple.

<u>Variables de sortie</u>

- **ret (le seuil retenu)** : Dans ce cas, il vaudra 120. C'est surtout utile quand on utilise des méthodes automatiques comme le Seuillage d'Otsu (présenté ci-dessous), où le meilleur seuil est calculé par l'algorithme lui-même.
- **image_seuil** : C'est l'image résultante, composée uniquement de pixels noirs et blancs.

- **Seuillage d'Otsu (automatique)** : L'algorithme calcule statistiquement le seuil optimal qui minimise la variance à l'intérieur des classes (fond et forme).

```
# Le paramètre 0 est ignoré car Otsu calcule lui-même le seuil
ret, image_otsu = cv2.threshold(image, 0, 255, cv2.THRESH_OTSU)
```

3. Filtrage

Le filtrage modifie la valeur d'un pixel en fonction de ses voisins via un masque (noyau). Il est utilisé pour réduire le bruit (Lissage) ou détecter les contours.

- **Filtre Moyenneur (Blur)** : Remplace chaque pixel par la moyenne de ses voisins.

```
# (5, 5) représente la taille du masque (kernel size)
image_floue = cv2.blur(image, (5, 5))
```

- **Filtre Gaussien** : Lissage plus doux qui préserve mieux les bords que le filtre moyen.

```
# 0 indique que l'écart-type est calculé automatiquement à partir de la taille du noyau (5,5)
image_gauss = cv2.GaussianBlur(image, (5, 5), 0)
```

- **Filtre Médian** : Très efficace contre le bruit de type "poivre et sel". Il préserve bien les contours nets.

```
# 5 représente l'ouverture du filtre (doit être un entier impair)
image_median = cv2.medianBlur(image, 5)
```

4. Opérations géométriques

Les opérations géométriques sont des transformations qui modifient la grille spatiale des pixels.

- **Rotation** : Elle nécessite le calcul d'une matrice de rotation ***M***, en suivant les étapes présentées ci-dessous :

```
(h, w) = image.shape[:2]
centre = (w // 2, h // 2)
# 45 est l'angle, 1.0 est l'échelle (zoom)
M = cv2.getRotationMatrix2D(centre, 45, 1.0)
image_tournee = cv2.warpAffine(image, M, (w, h))
```

  - Récupération de la hauteur *h* et la largeur *w* de l'image.
  - Calcul des coordonnées du point central de l'image. La division entière // est utilisée car les coordonnées des pixels doivent être des nombres entiers.
  - Génération d'une matrice mathématique 2 x 3 qui contient les instructions pour la rotation :
    - **centre** : Le point fixe autour duquel l'image va tourner.
    - **45** : L'angle de rotation en degrés (sens anti-horaire).
    - **1.0** : Le facteur d'échelle. À 1.0, l'image garde sa taille originale. Si on met **0.5**, l'image sera réduite de moitié pendant la rotation.

- Transformation :
  - **warpAffine** : Cette fonction prend chaque pixel de l'image d'origine et le déplace vers sa nouvelle position en utilisant la matrice M.
  - **(w, h)** : Indique la taille de l'image finale. En gardant la taille d'origine, les coins de l'image qui "dépassent" après la rotation de 45° seront coupés.

❖ **Extraction (Cropping)** : Utilise le découpage (slicing) des tableaux NumPy.

```
# image[y_debut:y_fin, x_debut:x_fin]
image_coupee = image[50:200, 100:300]
```

❖ **Redimensionnement :** On redimensionne une image (zoom ou réduction) en utilisant une technique mathématique avancée appelée l'interpolation bicubique.

```
# INTER_CUBIC est recommandée pour les agrandissements (plus de détails)
image_agrandie = cv2.resize(image, (nouvelle_w, nouvelle_h),
interpolation=cv2.INTER_CUBIC)
```

II. Développement de l'application d'amélioration d'image

L'interface à développer comporte les composants interactifs suivants :

- **Sélection et affichage initial :**
  - **Bouton « Parcourir » :** permet de parcourir les fichiers images (*.jpg, *.jpeg et *.png) et de sélectionner une image.
  - **Zone « Image originale » :** affiche l'image importée.
  - **Zone « Histogramme de l'image originale » :** visualise la distribution statistique des intensités de l'image source.
- **Égalisation d'histogramme :**
  - **Bouton « Appliquer » :** permet d'applique l'opération d'égalisation d'histogramme sur l'image sélectionnée et d'afficher l'image résultante ainsi que son histogramme dans les zones correspondantes, intitulées respectivement « **Image égalisée** » et « **Histogramme de l'image égalisée** ».
  - **Zone « Image égalisée » :** Affiche le résultat de l'image égalisée.
  - **Zone « Histogramme de l'image égalisée » :** Présente l'histogramme de l'image traitée.
- **Seuillage :**
  - **Deux boutons radios :** « **Seuillage binaire** » et « **Seuillage d'Otsu** », permettant de choisir la méthode de seuillage à appliquer.
  - **Bouton « Valider » :** Applique la binarisation et affiche l'image résultante (noir et blanc).

- **Filtrage :**
  - **Trois boutons radios :** permettent de choisir entre trois types de filtre : **Gaussien** (lissage doux), **Moyenneur** (flou uniforme) ou **Médian** (réduction de bruit impulsionnel).
  - **Bouton « Valider » :** applique le filtrage choisi et met à jour la zone d'affichage dédiée à l'image filtrée.
- **Opérations géométriques :**
  - **Trois boutons radios :** permettent de choisir entre trois opérations géométriques : **Rotation**, **Extraction** d'une partie de l'image ou **Agrandissement**.
  - **Bouton « Valider » :** applique l'opération choisie et l'affiche dans la zone.

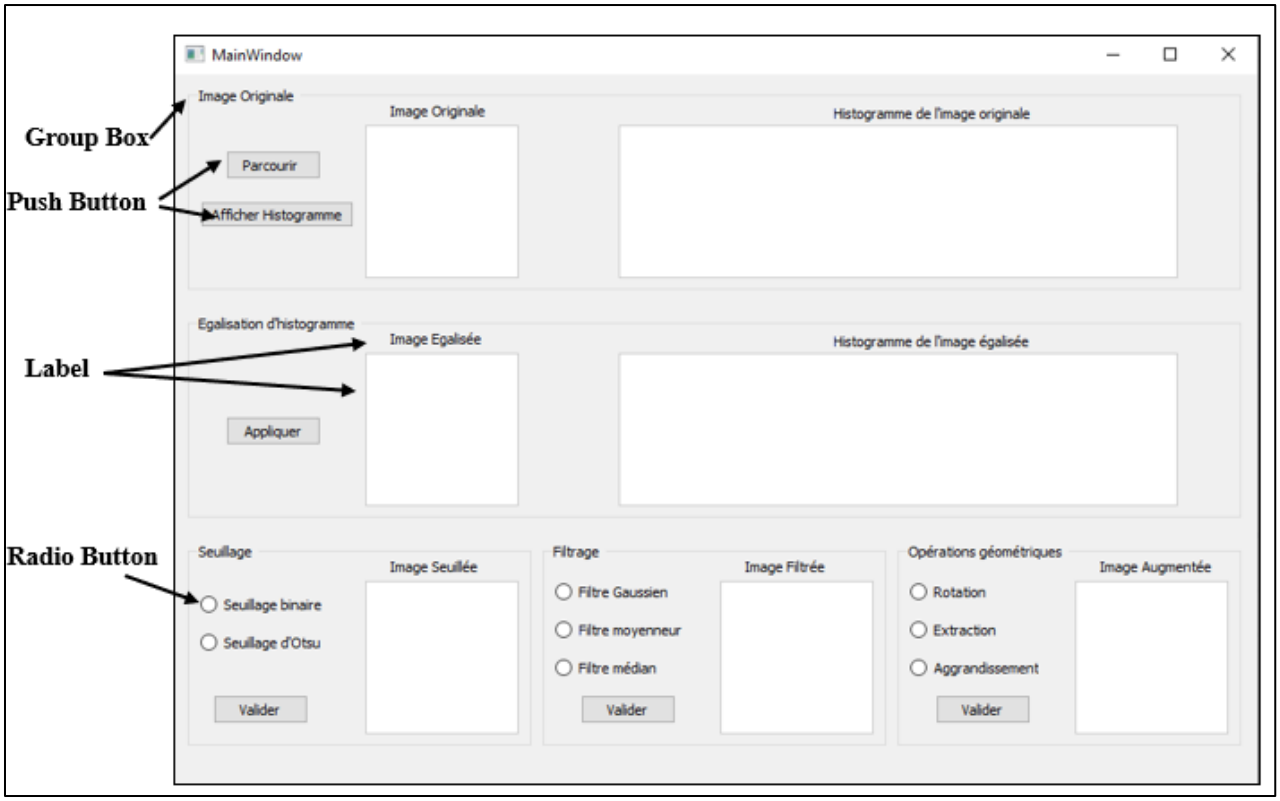

Figure 1. Aperçu de l'interface demandée

Travail demandé

1. **Création de l'interface graphique**
   a. **Création du projet** : Sous **PyCharm**, créez un nouveau projet nommé **« TP2 »**.
   b. **Design de l'interface** : À l'aide de **QtDesigner**, créez une fenêtre de type « MainWindow ». Modifiez-la pour contenir les éléments graphiques présentés en dessus (Figure 1).
   c. **Sauvegarde** : Enregistrez le fichier sous le nom « **design.ui** ».

d. **Génération du code Python** : Convertissez le fichier « design.ui » en code Python nommé « **design.py** ».

2. Affichage de l'interface graphique créée

Pour afficher l'interface graphique et vérifier le bon fonctionnement de l'application, suivez les étapes suivantes dans le fichier « **main.py** » :

a. Importation des bibliothèques nécessaires

```
from PyQt5 import QtWidgets, uic, QtGui
from PyQt5.QtWidgets import QFileDialog
from PyQt5.QtGui import QPixmap
import cv2
import sys
```

b. Chargement dynamique de l'interface

```
# Définition du fichier UI et chargement automatique des classes
qtcreator_file = "design.ui"
Ui_MainWindow, QtBaseClass = uic.loadUiType(qtcreator_file)
```

c. Création de la classe principale « DesignWindow » avec le développement de sa méthode d'initialisation

```
class DesignWindow(QtWidgets.QMainWindow, Ui_MainWindow):
    def __init__(self):
        super(DesignWindow, self).__init__()  # Initialisation
de la classe parente
        self.setupUi(self)  # Configuration des composants de
l'interface
```

d. Ajout de la fonction principale (Main)

```
if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    window = DesignWindow()
    window.show()
    sys.exit(app.exec_())
```

3. Implémentation de la logique de l'application

3.1. Importation et analyse initiale

a. Développez une fonction utilitaire « ***makeFigure()*** » permettant d'ajouter et d'afficher une image au sein d'un Widget de l'interface.

b. Implémentez une fonction « ***get_image()*** » qui permet de parcourir les fichiers système pour sélectionner une image (.jpg, .jpeg, .png), et d'afficher sa version en niveaux de gris ainsi que son histogramme dans les Widgets respectifs (**OriginalImg** et **OriginalHist**).

c. Calculez l'histogramme de l'image en niveaux de gris à partir de la fonction « ***show_HistOriginal()*** » qui permet aussi de générer et enregistrer la représentation graphique sous le nom « **Original_Histogram.png** ».

d. Dans la méthode « ***__init__*** », liez l'événement clic du bouton « parcourir » (**Browse**) à la fonction ***get_image()*.**

3.2. Égalisation d'histogramme

a. Développez la fonction « ***show_ImgHistEqualized()*** » qui permet de :

i. Appliquer l'égalisation d'histogramme.

ii. Sauvegarder l'image résultante « **Equalized_Image.png** » et l'afficher dans le Widget **EqualizedImg**.

iii. Calculer, enregistrer « **Equalized_Histogram.png** » et afficher l'histogramme de l'image égalisée dans le Widget **EqualizedHist**.

b. Dans la méthode « ***__init__*** », liez le bouton « Appliquer » (**Apply**) à la fonction ***show_ImgHistEqualized().***

3.3. Seuillage

a. Développez la fonction « ***show_ImgThresholding()*** » qui permet de :

i. Appliquer la méthode de seuillage choisie par les boutons radios sur la version en niveaux de gris de l'image sélectionnée.

ii. Enregistrer l'image résultante sous le nom « **Thresholding_Image.png** » et l'afficher dans le Widget dédié (**ThresholdingImg**).

b. Dans la méthode « ***__init__*** », liez le bouton « Valider » (**Validate_1**) à la fonction ***show_ImgThresholding().***

3.4. Filtrage

a. Développez la fonction « ***show_ImgFiltered()*** » qui permet de :

i. Appliquer un type de filtre choisi à travers les boutons radios sur la version en niveaux de gris de l'image sélectionnée.

Les caractéristiques des filtres sont les suivantes :

- **Filtre moyenneur** : utiliser un noyau de taille 11 × 11.

- **Filtre gaussien** : utiliser un noyau de taille 15 × 15 avec un écart-type égal à 10.
- **Filtre médian** : utiliser une taille de filtre égale à 13.

ii. Enregistrer l'image filtrée sous le nom « **Filtered_Image.png** » et l'afficher dans le Widget dédié (**FilteredImg**).

b. Dans la méthode « ***__init__*** », liez le bouton « Valider » (**Validate_2**) à la fonction ***show_ImgFiltered().***

3.5. Opérations géométriques

a. Développez la fonction « ***show_ImgAugmented()*** » qui permet de :

i. Appliquer les trois opérations géométriques choisies via les boutons radios sur la version en niveaux de gris de l'image sélectionnée :

- Rotation de 45° autour du centre de l'image. L'image de sortie doit conserver les dimensions originales.
- Extraction (Cropping) : Réaliser une découpe de l'image en extrayant uniquement le quart supérieur gauche (50% de la hauteur et 50% de la largeur). Cette opération doit illustrer une perte de contexte spatial sans changement d'échelle.
- Zoom Numérique (Resizing aléatoire) :
  - Générer un facteur d'échelle aléatoire s compris entre 1.5 et 4.0.
  - Agrandir l'image à l'aide d'une interpolation cubique pour préserver la qualité des détails.
  - Extraire la zone centrale de l'image agrandie pour revenir aux dimensions H x W initiales, créant ainsi un effet de "zoom" visuel.

ii. Enregistrer l'image augmentée sous le nom « **Augmented_Image.png** » et l'afficher dans le Widget dédié (**AugmentedImg**).

b. Dans la méthode « ***__init__*** », liez le bouton « Valider » (**Validate_3**) à la fonction ***show_ImgAugmented().***

4. Exécution de l'application

Exécutez votre application et vérifiez son bon fonctionnement.

Figure 2. Aperçu de l'interface après l'exécution de toutes les fonctionnalités développées